LAB University of Applied Sciences

Integration tests

LAB University of Applied Sciences
Bachelor of Engineering Industrial Information Texhnology
2027
Davin Pihlajasaari

Statement about using AI

The author of this thesis, Davin Pihlajasaari, used the following AI tools during the preparation of the thesis:

ChatGPT

Purpose of use: Generating the code and comments.

Explanation of the use of the tool: Used ChatGPT to generate the code for the Node server and hex to RGB function and had it aid me in making the tests. I also had it help with the comments on the code.

Responsibility

The author of this thesis, Davin Pihlajasaari, takes full responsibility for the content of this thesis and has reviewed and edited the content generated by the possible use of AI tools. The author has saved all discussions with AI applications during the thesis process. Thus, the extent of the author's own work and the use of AI can be verified if necessary.

The authenticity of this thesis has been checked with the Turnitin similarity checking software.

Abstract

Author(s)	Type of publication	Published in (year)
Davin Pihlajasaari	Task	2026
	Number of pages	
	6	

Title of the publication

Integration tests

Bachelor of engineering Industrial Information Technology

Petri Rantanen

Abstract

A report on making and documenting the process of an integration test.

Keywords

API, NodeJS, Mocha, Integration test, Unit test

Contents

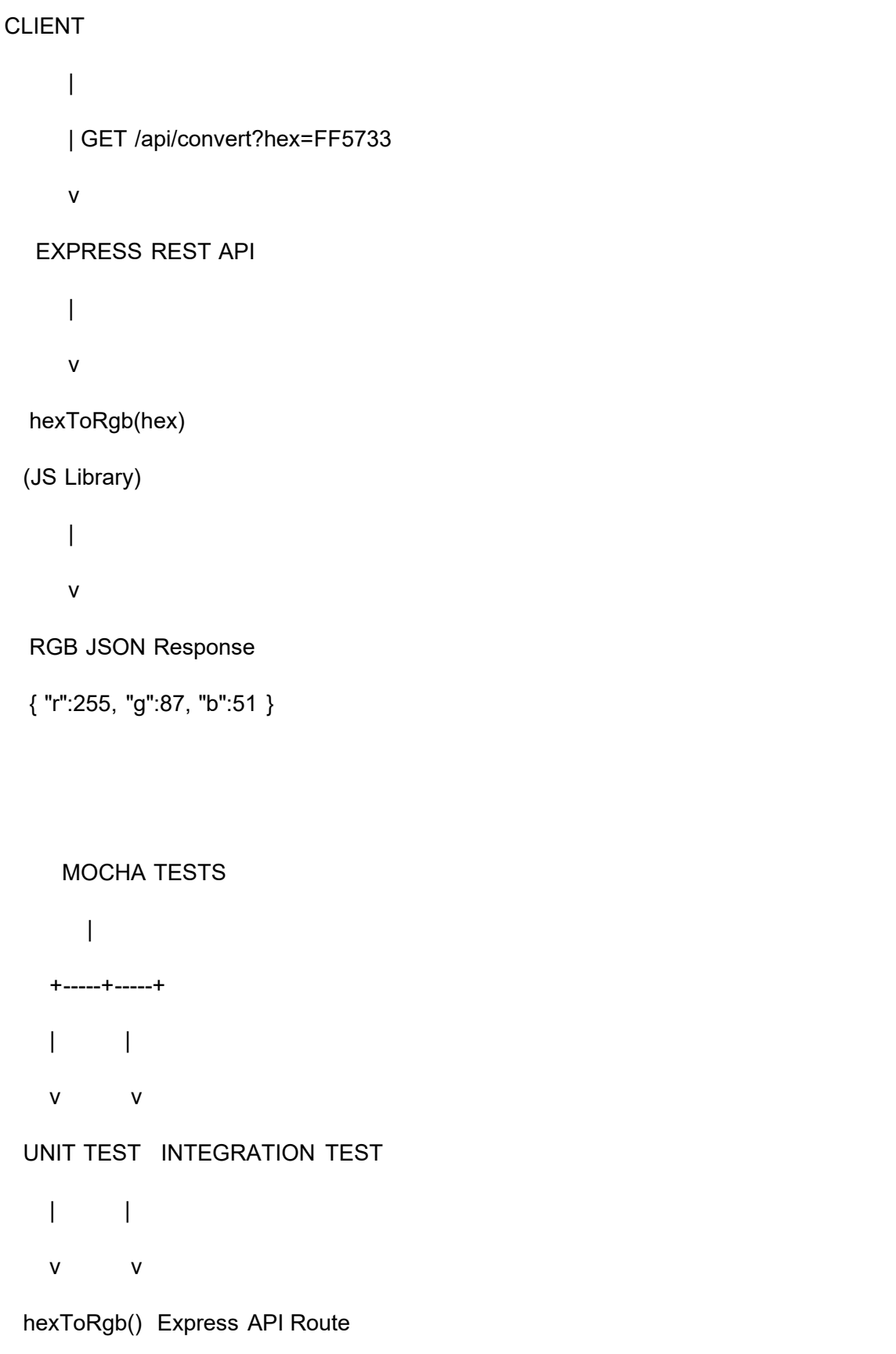
1 The system 1
2 The tests 2
2.1 Unit tests 2
2.2 Integration tests 3
2.3 Passing tests 4
3 Comparison 5
3.1 Tests in code 5
3.2 Tests in Postman or curl 5
References 6

Appendix 1. Appendix title

Appendix 2. Appendix title

1 The system

The system looks something like this:



2 The tests

2.1 Unit tests

✓ converts a six-digit hex color to RGB

✓ accepts hex without the # prefix

✓ supports three-digit shorthand

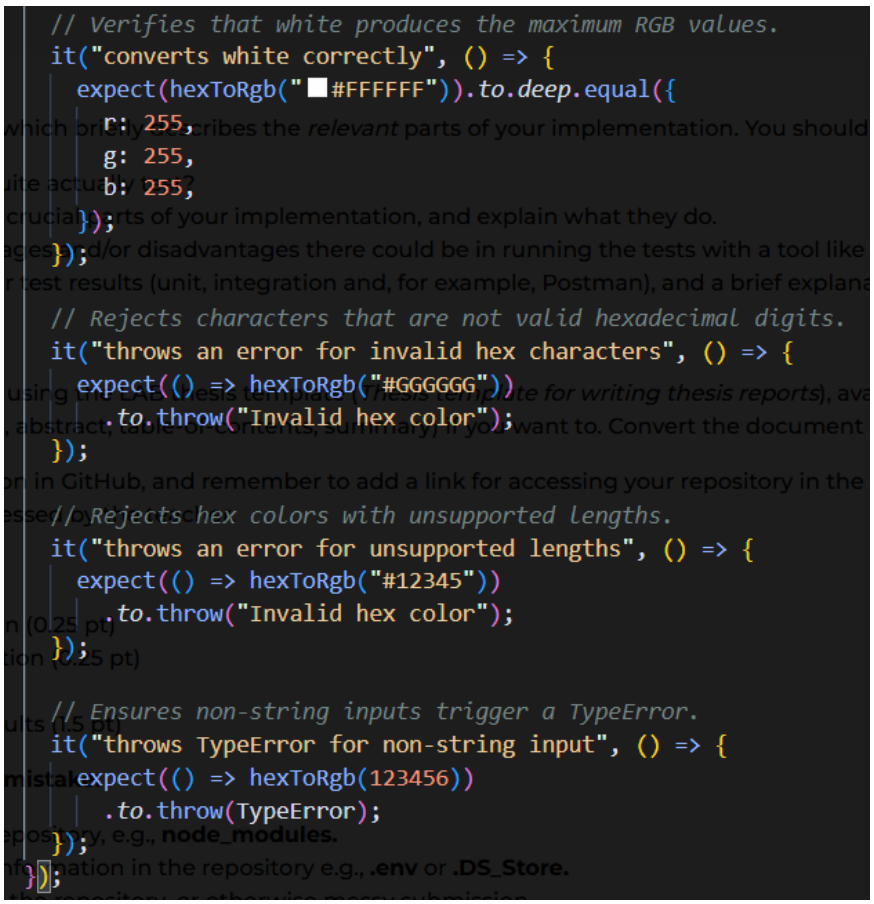
✓ converts black correctly

✓ converts white correctly

✓ throws an error for invalid hex characters

✓ throws an error for unsupported lengths

✓ throws TypeError for non-string input



```
// Verifies that white produces the maximum RGB values.
it("converts white correctly", () => {
  expect(hexToRgb("#FFFFFF")).to.deep.equal({
    r: 255,
    g: 255,
    b: 255,
  });
});

// Rejects characters that are not valid hexadecimal digits.
it("throws an error for invalid hex characters", () => {
  expect(() => hexToRgb("#GGGGGG"))
    .to.throw("Invalid hex color");
});

// Rejects hex colors with unsupported lengths.
it("throws an error for unsupported lengths", () => {
  expect(() => hexToRgb("#12345"))
    .to.throw("Invalid hex color");
});

// Ensures non-string inputs trigger a TypeError.
it("throws TypeError for non-string input", () => {
  expect(() => hexToRgb(123456))
    .to.throw(TypeError);
});
});
```

The above picture shows the four types of tests done.

My unit tests test for both three-character hex codes and six-character hex codes along with making sure that it checks for valid lengths and valid hex codes and string formats.

2.2 Integration tests

```
// Verifies that a valid hex color returns the expected RGB values.
it("returns RGB values for a valid hex color", async () => {
  const response = await request(app)
    .get("/api/convert")
    .query({ hex: "#FF5733" });

  expect(response.status).to.equal(200);
  expect(response.body).to.deep.equal({
    hex: "#FF5733",
    rgb: {
      r: 255,
      g: 87,
      b: 51,
    },
  });
});
```

✓ returns RGB values for a valid hex color

```
// Ensures three-digit shorthand colors are supported.
it("supports shorthand hex values", async () => {
  const response = await request(app)
    .get("/api/convert")
    .query({ hex: "F53" });

  expect(response.status).to.equal(200);
  expect(response.body.rgb).to.deep.equal({
    r: 255,
    g: 85,
    b: 51,
  });
});
```

✓ supports shorthand hex values

```
// Checks that invalid hex colors return a 400 status.
it("returns 400 for an invalid hex color", async () => {
  const response = await request(app)
    .get("/api/convert")
    .query({ hex: "XYZ" });

  expect(response.status).to.equal(400);
  expect(response.body).to.deep.equal({
    error: "Invalid hex color",
  });
});
```

✓ returns 400 for an invalid hex color

```
// Ensures a missing hex parameter returns a 400 status.
it("returns 400 when hex is missing", async () => {
  const response = await request(app)
    .get("/api/convert");

  expect(response.status).to.equal(400);
  expect(response.body).to.deep.equal({
    error: 'Query parameter "hex" is required',
  });
});
```

✓ returns 400 when hex is missing

My integration tests check for valid hex codes including shorthand hex codes (eg. F56), it also checks if the input string is a hex code in the first place and also checks if the input is empty.

2.3 Passing tests

```
GET /api/convert:
  ✓ returns RGB values for a valid hex color (72ms)
  ✓ supports shorthand hex values
  ✓ returns 400 for an invalid hex color
  ✓ returns 400 when hex is missing

Converter Test:
  ✓ converts a six-digit hex color to RGB
  ✓ accepts hex without the # prefix
  ✓ supports three-digit shorthand
  ✓ converts black correctly
  ✓ converts white correctly
  ✓ throws an error for invalid hex characters
  ✓ throws an error for unsupported lengths
  ✓ throws TypeError for non-string input


12 passing (110ms)
```

The above image shows the unit and integration tests passing.

3 Comparison

3.1 Tests in code

Tests in code allow you to make generic tests that need to be done every time you make an update to the code to see if you have broken the code in some way.

3.2 Tests in Postman or curl

Doing tests in Postman or curl allow you to do one off tests while you are testing a new function or API route before it gets added to the main code as you don’t want to do every test each and every time.

References

https://github.com/DavinPihlajasaari/Devops_engineering/tree/main/Petri/integration-test-task